

# claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

## Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-a91e44d9d600eaba7.jsonl`
- SHA-256: `6aa573fd1e1cd425cf18a21ff69782df39df3952f6ecf44c5571e693316a4eea`
- Source modified: `2026-06-21T13:55:19+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

## Transcript

### 1. user (2026-06-21T13:52:09.109Z)

Research Docker / docker-compose as the "batteries-included" unit for an ffmpeg + (dual) torch + FastAPI + worker + SQLite app. Cover: multi-stage builds; putting TWO python envs in one image (main torch 2.6+cu124 AND a WASB conda torch 1.11+cu113 env via micromamba) vs separate service containers; image size realities; GPU passthrough (nvidia-container-toolkit / --gpus all) vs CPU-only image; Docker Desktop on Windows for a non-tech user (friction!); compose for server+worker+bundled-SPA; volumes for the user's videos + SQLite + weights. We already have deploy/cloudrun/Dockerfile. Use web search.

Return the RESEARCH schema. Context about our app:

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.  
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).  
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing),  
C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:

```
- pyproject.toml: hatchling build, namespace package (no backend/__init__.py),
packages=["backend","training"], console script: indexer = backend.main:main. requires-python
>=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT),
reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai
is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker
{infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter
{gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda,
torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps.
Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile +
compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-
serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry);
backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job);
backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi).
```

The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.

- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP\_APP\_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).

khelsutra/docs/LOCAL\_APPLIANCE\_ARCHITECTURE.md is the live appliance design.

Always ground claims in file:line or a doc path. Read the actual files; do not assume.

## 2. assistant (2026-06-21T13:52:11.822Z)

I'll research Docker/docker-compose as the packaging unit for this app. Let me first ground myself in the actual repo state, then do targeted web searches.

## 3. user (2026-06-21T13:52:13.216Z)

```
1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33     RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34         && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
```

```

36      && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37      && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38      && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39      && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40          torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41      && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
42      && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43          hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
44
45      # --- Main application env ---
46      WORKDIR /app
47      COPY pyproject.toml README.md ./
48      COPY backend ./backend
49      COPY training ./training
50      RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52      # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
53      # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54      ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55          WASB_REPO_DIR=/opt/models/WASB-SBDT \
56          WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57          RALLY_DETECTOR_IMPL=native
58
59      # The dispatcher (CloudRunCompute) supplies the per-execution args:
60      #   infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
61      ENTRYPOINT ["python3", "-m", "backend.worker"]
62

```

#### 4. user (2026-06-21T13:52:13.392Z)

```

deploy\digitalocean\bootstrap.sh
deploy\digitalocean\mount_scratch.sh
deploy\digitalocean\gpu_setup.sh
deploy\gcp\teardown.sh
deploy\digitalocean\README.md
deploy\gcp\README.md
deploy\cloudrun\Dockerfile
deploy\cloudrun\README.md
deploy\gcp\provision.sh
deploy\gcp\ops-agent-startup.sh
deploy\gcp\create_gpu_vm.sh

```

#### 5. user (2026-06-21T13:52:14.280Z)

No files found

#### 6. user (2026-06-21T13:52:14.305Z)

```

deploy\cloudrun\Dockerfile

```

#### 7. assistant (2026-06-21T13:52:17.208Z)

Let me read the live appliance design doc and key engine files in parallel, then start web searches.

#### 8. user (2026-06-21T13:52:18.214Z)

## 9. user (2026-06-21T13:52:19.113Z)

```
1      #!/usr/bin/env bash
2      #
3      # Native (no-WSL) WASB shuttle-detector setup for a Linux GPU box ? the M3.5 enabler for
4      # real trajectory extraction. Replicates docs/TRACKNET_WSL_SETUP.md's WSL conda env
5      # *natively* on Linux. Run AFTER ./bootstrap.sh --gpu (which provides CUDA torch for the
6      # main venv); this builds a SEPARATE conda env for WASB (torch 1.11) ? never co-
installed
7      # with the indexer's torch (framework conflict).
8      #
9      # Requires an NVIDIA GPU + driver (the D0 `gpu-*-base` image ships it).
10     #
11     # ? WASB-C3: the pretrained badminton weights' license / source-dataset provenance is
NOT
12     # verified here. Fine for internal extraction + Gen-0 bootstrapping; clear it before
sharing
13     # any derived trajectories or model. See docs/DECOUPLED_COMPUTE/06.
14     set -euo pipefail
15
16     MODELS_DIR="${RALLY_MODELS_DIR:-$HOME/models}"
17     CONDA_DIR="${CONDA_DIR:-$HOME/miniconda3}"
18     WASB_REPO="$MODELS_DIR/WASB-SBDT"
19     mkdir -p "$MODELS_DIR"
20
21     echo "[gpu] [1/5] miniconda (native Linux)"
22     if [ ! -x "$CONDA_DIR/bin/conda" ]; then
23         curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh
24         bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh
25     fi
26     # shellcheck disable=SC1091
27     source "$CONDA_DIR/etc/profile.d/conda.sh"
28
29     # Modern conda BLOCKS non-interactive `conda create` until the defaults-channel Terms of
Service
30     # are accepted (CondaToSNonInteractiveError). Accept them so the env build doesn't fail
on a fresh
31     # box. (Observed on a GCP Deep-Learning-VM image, 2026-06-20.)
32     conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
>/dev/null 2>&1 || true
33     conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
>/dev/null 2>&1 || true
34
35     echo "[gpu] [2/5] WASB-SBDT repo"
36     if [ ! -d "$WASB_REPO/.git" ]; then
37         git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"
38     fi
39
40     echo "[gpu] [3/5] conda env 'wasb' (py3.8 + torch 1.11.0+cu113 ? the verified WASB
stack)"
41     conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.8
42     conda activate wasb
43     python -m pip install -q torch==1.11.0+cu113 torchvision==0.12.0+cu113 \
44         --extra-index-url https://download.pytorch.org/whl/cu113
45     [ -f "$WASB_REPO/requirements.txt" ] && python -m pip install -q -r
"$WASB_REPO/requirements.txt" || true
46     # WASB-SBDT's requirements.txt is INCOMPLETE: src/ imports hydra, pandas, etc. that it
omits, so
47     # `import wasb_infer` deps fail without these. (Found on the GCP L4 port, 2026-06-20.)
numpy is
48     # pinned <1.24 for torch-1.11 ABI compatibility.
49     python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy
```

```

tqdm pyyaml matplotlib "numpy<1.24" || true
50
51     echo "[gpu] [4/5] pretrained badminton weights"
52     WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
53     if [ -f "$WEIGHTS" ]; then
54         echo "[gpu]         found $WEIGHTS"
55     else
56         echo "[gpu]         !! MISSING $WEIGHTS ? fetch wasb_badminton_best.pth.tar
(Drive/bucket) before extraction."
57     fi
58
59     echo "[gpu] [5/5] GPU sanity check (in the wasb env)"
60     python - <<'PY'
61     import torch
62     ok = torch.cuda.is_available()
63     print("[gpu]         torch", torch.__version__, "| cuda_available", ok,
64           "| device", (torch.cuda.get_device_name(0) if ok else "CPU"))
65     assert ok, "CUDA not available in the wasb env ? check the GPU driver / base image"
66     PY
67
68     echo "[gpu] native WASB env ready.  Activate with:  source
$CONDA_DIR/etc/profile.d/conda.sh && conda activate wasb"
69     echo "[gpu] (the LinuxNativeRunner ? backend/pipeline/detectors ? drives extraction once
it lands; M3.5.)"
70

```

## 10. user (2026-06-21T13:52:19.540Z)

```

1     # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2     #
3     # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4     # (no `backend/__init__.py`; mpy.ini sets explicit_package_bases for the same
5     # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6     # auto-detection would otherwise miss the un-__init__'d `backend` root.
7     #
8     # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9     # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10    # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11    # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12    [build-system]
13    requires = ["hatchling"]
14    build-backend = "hatchling.build"
15
16    [project]
17    name = "badminton-highlight-indexer"
18    version = "0.1.0"
19    description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20    readme = "README.md"
21    requires-python = ">=3.10"
22    license = { text = "MIT" }
23    authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24    keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26    # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27    dependencies = [
28        "fastapi>=0.111.0",
29        "uvicorn>=0.30.1",
30        "opencv-python>=4.9.0.80",
31        "numpy>=1.26.4",
32        "pydantic>=2.7.1",
33        "jinja2>=3.1.4",
34        "python-dotenv>=1.0.0",
35        "google-genai>=2.4.0",
36        "supervision>=0.21.0,<0.30.0",

```

```

37 ]
38
39 [project.optional-dependencies]
40 # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41 dev = [
42     "pytest",
43     "pytest-cov",
44     "httpx",          # fastapi TestClient transport
45     "ruff",
46     "mypy",
47 ]
48
49 # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50 # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51 # fetched over SSH; if you lack access, simply omit this group ? reporting
52 # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53 # pip install -e ../sports-obsreport --no-build-isolation
54 reporting = [
55     "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56 ]
57
58 # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59 # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60 # express, so they are NOT declarable here. Install them out-of-band instead:
61 # # CUDA 12.4 box:
62 # pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63 # # CPU-only box (also what CI uses):
64 # pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
65 gpu = []
66
67 # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68 # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
69 # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
70 # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
71 # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
72 storage-s3 = ["boto3>=1.34"]
73 storage-gcs = ["google-cloud-storage>=2.16"]
74
75 [project.scripts]
76 # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
77 indexer = "backend.main:main"
78
79 [tool.hatch.metadata]
80 # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
81 # reference*); hatchling requires opting in to allow those in metadata.
82 allow-direct-references = true
83
84 [tool.hatch.build.targets.wheel]
85 # `backend` is a namespace package (no __init__.py); list both import roots
86 # explicitly so an editable/wheel install exposes `import backend` and
87 # `import training` regardless of cwd.
88 packages = ["backend", "training"]
89

```

## 11. assistant (2026-06-21T13:52:22.308Z)

Now let me read the live appliance design doc and check the worker/main entry points.

## 12. user (2026-06-21T13:52:23.531Z)

```

1 # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2

```

```

3      The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4      dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5      a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7      > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8      > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9      > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10
11     ## 0. Prerequisites (owner)
12     - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13     - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14     `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15     - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17     ## 1. Build + push the image (Artifact Registry)
18     ```bash
19     PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
20     gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
21     gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest -f
deploy/cloudrun/Dockerfile .
22     ```
23     *(A CUDA + conda image is large; the first build is slow. Budget-watch per D17.)*
24
25     ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
26     ```bash
27     gcloud run jobs create rally-worker \
28     --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
29     --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
30     --max-retries 0 --task-timeout 3600 \
31     --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
32     # (stage the weights into the container at start, or fetch from gs:// in an init step ?
WASB-C3.)
33     ```
34
35     ## 3. Wire the dispatcher (control plane)
36     The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
37     `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
38     ```bash
39     export DEPLOYMENT_PROFILE=cloud-serving
40     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
41     # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
42     # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
43     python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
44     ```
45
46     ## 4. M7 acceptance (the owner runlog)
47     Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
48     `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
49     rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test

```

```
(`ffmpeg -f null -`
50     exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
51
52     ## Gotchas
53     - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
54     + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55     - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
56     ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
57     - A **failed** Job currently writes **no** marker ? the dispatcher's bounded poll times
out
58     (`ExecutionPolicy.max_wait_sec`) and `_dispatch_remote` returns a clean **rc 4** (not
a traceback).
59     A failure-marker (carrying the real non-zero rc, so the dispatcher distinguishes
*failed* from
60     *hung/cold-start-slow*) is a follow-up.
61     - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
62
```

### 13. user (2026-06-21T13:52:23.692Z)

```
1     # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3     > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-20
4     > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5     > **Tracking anchors:** $9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6     > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7     > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9     ---
10
11     ## 0. TL;DR
12     **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
**one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the
degenerate collapse of the same storage/worker split.
13
14     The **single most important caveat:** the engine has **zero app-layer auth and CORS
`***` `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays
bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`,
and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the
tenancy boundary.**
15
16     Two honest hardware truths: (1) the current droplet is **CPU-only** ? real detection
there is **Gemini (cloud)** or **stub (CPU mock)**; **real local-GPU detection needs a GPU
host** (native WASB, in-flight). (2) The owner priority is to **stand up a GPU host ASAP** ?
provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs **in parallel**
with the UI MVP, which ships on Gemini today.
17
18     ## 1. Problem & goal
```



```

19     The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker
{infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the
FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified
backend/main.py]`. What's missing is a console that lets a non-CLI operator run the whole
loop on a single box, honestly, behind a gate.
20
21     Headline CUJ (owner's words): a hosted website where I pick an input ? a
Google Drive file, a bucket file (`s3:///gs://`), or a local upload ? it lands in the
bucket (DO Spaces / GCS) or locally ? I generate highlights on a GPU-enabled host
running the real e2e ? I pick rallies, watch, and download ? completely from the UI.
22
23     Outcome ("good"): an operator signs in, picks a source, watches it index, curates
rallies, downloads a reel ? on the live CPU droplet today (Gemini/stub), and on a GPU host
later (native WASB) with no UI fork.
24
25     Read to get current: `PER_RALLY_SELECTION.md` ($3/$4/$10 contract anchors), engine
`HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built),
`VIDEO_LOCALITY_MODEL.md` (privacy/retention).
26
27     ## 2. Decisions locked
28     | # | Decision | Source / date | Implication |
29     |---|---|---|---|
30     | D1 | Reverse-proxied single origin; engine stays `127.0.0.1:8000` | `[verified
main.py:638]`; 2026-06-20 | CORS `` is moot from the browser ? the proxy is the boundary |
31     | D2 | Edge auth, zero engine auth code | `HOSTING_PLAN.md`; 2026-06-20 | Caddy
basicauth OR cloudflared + Cloudflare Access |
32     | D3 | MVP drives the in-process FastAPI worker loop, `storage=local` | `[verified
job_worker.py:37, local.py:33]`; 2026-06-20 | bucket worker = reserved scale path, out of MVP |
33     | D4 | Embed RallyPicker from `PER_RALLY_SELECTION.md` verbatim | PR [#11];
2026-06-20 | the console is a superset; do not redesign the picker |
34     | D5 | Console lives in the khelsutra web React scaffold | `web/README.md`;
2026-06-20 | Cloudflare-Pages-style root, not engine static |
35     | D6 | Compute placement is a setting, rendered from `GET /api/capabilities` |
`[design]`; 2026-06-20 | UI offers ONLY what the box can honestly do |
36     | D7 | Ingest is a source menu (local upload · bucket URI · Drive) normalizing to
one video object | `[verified backend/storage/ref.py parse_uri]`; 2026-06-20 | Drive is a
stub today ? scoped honestly ($4.2) |
37     | D8 | GPU host is provider-agnostic (DO droplet / GCE VM / Cloud Run+GPU); storage
stays the bucket | Owner 2026-06-20 | compute and storage are separate planes; no provider lock-
in |
38     | D9 | Observability is first-class ? structured logs on every new path + a DoD item
| Owner 2026-06-20 | see $5; verified during CUJ replay |
39     | D10 | Delivery = a self-hosted webserver + UI the user spins up (NOT a desktop
app); the user points at local storage + GPU/CPU OR cloud locations and the UI drives the
tool | Owner 2026-06-20 | supersedes `DESKTOP_APP_PLAN.md`; one artifact (this appliance)
runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
40
41     ## 3. Foundation ? what the engine gives us today `[verified this session unless
noted]`
42     - FastAPI loop: `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ?
`POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}`
(FileResponse) `[main.py]`. Chunked upload exists ? `POST /api/upload/init` · `PUT
/api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}`
`[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` never calls them (it only
hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is client-
wiring only, no engine change.
43     - In-process worker: one module-global `ThreadPoolExecutor(max_workers=1)`
`[verified job_worker.py:37]` ? strictly one job at a time; the DB `jobs` table is the
clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a
separate knob `indexing.ai_max_workers` (default 5 `[verified models.py:183]`; set to 1
to serialize) ? it is not already 1.
44     - Storage: `storage.backend` default = `local` `[verified models.py:298]`;
`LocalStorage.fetch_to_local` is identity `[verified local.py:33]`, `put` no-ops on
`src==dst` ? so on a single box the worker's pull?pipeline?push collapses to read-local/write-
local. URI scheme `parse_uri` `[verified ref.py]`: bare `local://` ? local; `s3://`

```

```

(AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs:///`gs:///` ; `gdrive:///` ? **follow-up stub
(emulated only)**. Credentials never in config ? boto3 default chain / `~/aws` `[verified
s3.py]`.
45 - **Compute seam:** `indexing.detector_impl` ? `{auto`=WSL, `native`=Linux-native,
`stub`=CPU mock}; default "auto" `[verified models.py:129]`, resolved in
`trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch
`feat/native-wasb-runner`:** `native_wasb_runner.py` **exists** `[verified]`;
**`fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified
fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified
:57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ?
not exercisable on the droplet or in the offline suite. Treat native as **not production-
verified**.
46 - **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ?
pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified
worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` +
`--set indexing.skip_ai_handoff=true` + `storage.s3`.
47 - **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access
email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET
/api/health` **now EXISTS** (engine M1, [#268]) ? `{status, sport, default_segmenter}`
`[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.
48 - **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts
the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the
full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md,
DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run
there.
49 - **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row
`get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any
**`Dockerfile`**, any per-video **cost ledger** `[verified ? grep returns none]`.
50
51 ## 4. Design / architecture
52
53 ### 4.1 Topology & request path
54 ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy /
static host** serving the `web/` SPA bundle and proxying `/api/*` ? engine **FastAPI
`127.0.0.1:8000`** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a
GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing
site stays a **separate** systemd unit on a **separate** subdomain.
55
56 ```
57 PUBLIC INTERNET
58 ? (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
59 ? cloudflared Tunnel + Cloudflare Access (no inbound ports) OR Caddy :443 +
basicauth
60 ?
61 ?????????????????????????????????????????????????????????????????????????????????
62 ? ONE BOX firewall BLOCKS inbound 8000 + 9000/9001/9023 ?
63 ? (2) reverse proxy / static host (Caddy/cloudflared) ?
64 ? /, /assets/* ? web/ React SPA /api/*, /api/highlights/* ? :8000 ?
65 ? ? ONE ORIGIN to the browser ? engine CORS '*' is irrelevant ?
66 ? ? ?
67 ? (3) ENGINE FastAPI (host=127.0.0.1:8000, HARD-bound [main.py:638]) ?
68 ? control: jobs table + ThreadPoolExecutor(max_workers=1) drain ?
69 ? data: uploads.py chunked upload · /api/compile · /api/highlights ?
70 ? ? ?
71 ? (4) STORAGE = LocalStorage (default) fetch=identity · put=no-op(src==dst) ?
72 ? ? ?
73 ? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet] ?
74 ? native WASB (GPU host) · auto (owner Win+WSL) ?
75 ?????????????????????????????????????????????????????????????????????????????????
76 RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
77 ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3:///` ?
outputs/<vid>/ back
78 (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)

```

```

79     ``
80
81     ### 4.2 Ingest sources (the headline) ? one menu, one normalized video object
82     All sources normalize to ""a video the pipeline can read"" ? a local path (MVP) or a
`StorageRef` (scale):
83
84     | Source | Path today | Status |
85     |---|---|---|
86     | **Local upload** | SPA chunks the file ? `max_part_mb` via the existing
`/api/upload/init\|part\|complete` `[verified uploads.py]` ? server path ? `POST /api/process`.
(Scale: also `StorageBackend.put` to `s3://\`gcs://\.`) | **wire client only** (endpoints exist)
|
87     | **Bucket URI** (`s3://\`, `gs://\`) | **POST /api/process` accepts the bucket URI
directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283
? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the
**engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs`
is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket
URI 400.)* | **engine: DONE (verified); SPA ingest call: design (= B-P2)** |
88     | **Google Drive** (`gdrive://\`) | `parse_uri` recognizes it, but the gdrive backend is
an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]` | **stub ? roadmap;** MVP
requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real |
89
90     ### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
91     | Placement | Runs | Needs | Real/mock | Honest caveat |
92     |---|---|---|---|---|
93     | **CPU droplet ? gemini** (MVP target) | `segmenter=gemini`, in-proc worker |
`GEMINI_API_KEY`; network; **no GPU** | **REAL** | Cloud brain: uploads ?1280px chunks to Google
? **must be in consent copy**; **no per-video cost ceiling** today |
94     | **CPU droplet ? stub** | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` |
nothing | **MOCK** | not real tracking ? UI must label "stub (CPU mock ? not real detection)";
demo/regression only |
95     | **CPU droplet ? motion** | `segmenter=motion` | nothing | REAL, low-quality | only no-
key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96     | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ?
`NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud
Run+GPU**; `WASB_WEIGHTS` | **REAL**, on-device | partially wired (fusion builds it, trajectory
refuses); **owner byte-parity gate**; **Cloud Run/GCE need a containerized worker ? no
Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97     | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU
| REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98     | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU
host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration
endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as
a setting" is **design** ? selector shown **disabled** in MVP |
99
100     **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit
for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the
**containerization feasibility check** above.
101
102     ### 4.4 Auth / exposure (non-negotiable invariant)
103     No engine auth, by design; the gate is external. **Invariant (all three must hold):**
engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist
`[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**;
set `security.allowed_video_root`; rate-limit; per-user quota.
104
105     ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)
106     - **S1 Ingest & Process** ? the source menu ($4.2) + a segmenter picker from
`/api/capabilities` ? `POST /api/process`.
107     - **S2 Progress** ? 3s poll of the free-text stage
(`hashing/validating/segmenting/finalizing`), a **"one job at a time"*** notice, and the restart-
failure terminal state.
108     - **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER_RALLY_SELECTION.md`):
honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter.
**Per-rally is ONE screen of the console.**
109     - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for

```

```

**reload-survival**.
110 - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target
selector shown DISABLED** ("single-box: local") until the bucket-worker bridge lands.
111
112 ### 4.6 Reuse map
113 **Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked
upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing
`NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the
`HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units;
`GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the
SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker
bridge.
114
115 ## 5. Observability ? logging & monitoring `[required, first-class]`
116 Launch-readiness directive (owner 2026-06-20): **logs and monitoring taken seriously,
with a full log-analysis pass across every new code path.** The engine already uses a
centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new
path.
117
118 - **Correlation ID end-to-end.** Thread the existing `job_id` (from `/api/process`)
**plus** an `upload_id`/`request_id` through ingest ? process ? jobs ? compile ? highlights, and
into the worker for the bucket path, so **one CUJ is traceable** across UI ? FastAPI ? worker ?
GPU host. Log it on every line.
119 - **Structured logs.** New endpoints/screens emit structured (JSON-able) logs at **entry
/ result / error** ? not just "it ran". The SPA logs client-side errors + failed fetches with
the same correlation id.
120 - **Loud failures, never silent drops.** Carry the engine's ethos (#240 "surface failed
chunks loudly") + RallyPicker's `min_shot_count`-warning rule: a partial/empty reel must
**never** look complete; every dropped clip/chunk is logged + surfaced in the UI.
121 - **Health + metrics.** Ship the missing `GET /api/health` `[HOSTING_PLAN.md:42]` and a
thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a
ledger exists) for the engine-mode banner + alerting.
122 - **Aggregation for transient hosts.** A scale-to-zero Cloud Run / GPU host is
**ephemeral** ? logs must ship to a durable sink (the bucket or a log service), not die with the
instance.
123 - **Log-analysis pass before merge (DoD item).** Every PR that adds a code path includes
a check that the path logs meaningfully, and the **logs are inspected during CUJ live-replay**
($8 Layers 1?3). Ties [[feedback-launch-quality-bar]].
124
125 ## 6. Risk table
126 | Risk | Today | Mitigation |
127 |---|---|---|
128 | No app auth + CORS `` `[main.py:42]` | open if any of {localhost-bind, firewall,
gate} slips | enforce all three; **LIVE posture test** that raw `:8000` is refused from outside
|
129 | Test ports on shared droplet | MinIO `9000/9001` + GCS emu `9023` | firewall closes
them |
130 | Retention | `output/`+`uploads/` originals **never deleted** | sweep / keep-proxy-only
before any non-owner exposure |
131 | Surprise Gemini/GPU bill | **no cost ceiling/ledger** | per-video cap;
`ai_max_workers` (default 5) tunable; `max_workers=1` job serialization |
132 | Native WASB unverified | partially wired; owner-parity-gated | treat as not-
production-verified until owner byte-parity on a real GPU box |
133 | New engine endpoints | `capabilities`/`videos`/`reels`/`health` don't exist |
coordinate engine PRs with active sessions; **branch from `origin/master`, re-verify HEAD,
explicit `git add`** |
134 | Drive ingest overstated | `gdrive://` is a stub | don't present Drive as live; require
Drive?bucket pre-step until real |
135 | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before
promising Cloud Run/GCE |
136 | Shared-checkout collisions | active worker sessions share the engine tree | the
branch-safety protocol on every engine-side PR |
137 | **Provisioning-credential leak** | a DO API token / cloud creds could leak if pasted
in chat or committed (cf. the **already-rotated Spaces key** exposed in chat,
`DECOUPLED_COMPUTE/HANDOFF.md`) | authenticate the CLI **on-box** (`doctl auth init` interactive

```

```

/ `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token,
revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action
every time** |
138
139     ## 7. Honest limits ? what this does NOT do
140     Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs
global). Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make
the remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source
playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not**
containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-
parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the
**console plumbing**, not engine rally-detection quality (a separate track).
141
142     ## 8. CUJs & live-test plan `[required]`
143     **CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core);
**CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters +
honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).
144
145     **Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:**
146     - **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing
the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure;
`query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ?
CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`).
The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start
dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel
link via snapshot/network). Catches reload-survival (CUJ-2), the `play_segments` field-name bug
(CUJ-3), capability honesty (CUJ-4).
147     - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main
--server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock
and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema,
compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock
handler **and** a real-engine contract test **in the same PR**. Runs in CI.
148     - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1
with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety
posture test** (raw `:8000`/`:900x` refused from outside) as an assertion, not an assumption;
reuse the `parity.yml` dual-host pattern.
149     - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job +
headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no
stacking**; coverage at/above the floor; **logs inspected during replay** ($5).
150     - **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU
box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/`cv2`-vs-`ffmpeg`
behavior; a multi-GB browser upload. Reported back into $9, stated plainly as **not verified**
in the offline suite.
151
152     ## 9. Deliverables & progress tracker ? **source of truth**
153     Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR
ships its tests + meaningful logs ($5).
154
155     | ID | Deliverable | Depends on | Gated? | Status | PR |
156     |----|-----|-----|-----|-----|----|
157     | T0 | This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers | ? | owner | ? |
(this PR) |
158     | T1 | Mock engine + capability fixtures + browser-replay CUJ scripts (L1) | T0 | no | ?
| ? |
159     | T2 | Appliance shell: Caddyfile + systemd units; firewall; **edge auth**; rotate key
(P1) | ? | owner | ? | ? |
160     | T3 | L3 live posture test (raw `:8000`/`:900x` refused from outside) | T2 | no | ? | ?
|
161     | T4 | S1 ingest (local upload wired) + segmenter picker | T1 | no | ? | ? |
162     | T5 | S2 progress (poll + one-job + restart-failure) | T4 | no | ? | ? |
163     | T6 | S3 embed **RallyPicker** + compile + download | T4 | no | ? | ? |
164     | T7 | *(engine)* `GET /api/capabilities` + `GET /api/health` + banner | T1 | coord | ?
| ? |
165     | T8 | *(engine)* `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival) |

```

```

? | coord | ? | ? |
166      | T9 | S5 Settings (config+capabilities; disabled compute/storage target) | T7,T8 | no |
? | ? |
167      | T10 | Observability pass: correlation id end-to-end + structured logs + metrics ($5) |
T4 | no | ? | ? |
168      | T11 | Retention sweep for `output/+`uploads/` | ? | owner | ? | ? |
169      | T12 | Ingest from bucket URI (s3://`gs://`) wiring ? engine half DONE
(`api/process` accepts URIs, M2/[#280], [verified]); remaining = the SPA ingest call (=
B-P2) | T4 | coord | ? | engine done; SPA pending |
170      | T13 | GPU host stand-up (provider-agnostic) + wire detector_impl=native`; owner
byte-parity (P4) | branch | ? owner | ? | ? |
171      | T14 | (reserved) Bucket-worker bridge ? remote-GPU "compute as a setting" (P5) |
T8,T12 | ? owner | ? | ? |
172
173      Owner priority (2026-06-20): T13 (GPU host) is elevated to run in parallel from the
start ? owner provisions the host + token; the engine session lands native WASB; the UI MVP
does not block on it (ships on Gemini).
174
175      ## 10. Open questions ? owner / external
176      Blocking gates: (1) edge-auth ? Caddy basicauth (self-contained) vs
cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) retention
defaults ? auto-delete originals after compile? keep proxies+intervals only? window?
(privacy/consent). (3) per-video Gemini/GPU cost ceiling ? what cap, hard-fail vs warn? (4)
GPU host choice ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker
worth building for Cloud Run)? (5) P5 bridge ownership ? who builds the orchestration
endpoint + how it coordinates with the active worker sessions.
177      Nice-to-knows: console home (web/ vs engine static); a review-playback streaming
endpoint; per-user scoping timing; containerization vs runbook.
178
179      ## 11. Definition of done
180      - [ ] CUJ-1 works end to end on the droplet (Gemini and stub) behind the gate, via
the ingest menu.
181      - [ ] Raw :8000/`900x` unreachable from outside (tested, not assumed).
182      - [ ] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped
honestly (not faked).
183      - [ ] RallyPicker embedded with honesty rules; min_shot_count` surfaced.
184      - [ ] `api/capabilities` + `api/health` + banner truthful; reload-survival via
`api/videos` + `api/reels`.
185      - [ ] Observability: correlation id threaded end-to-end; structured logs at every
new path; loud failures; logs inspected during CUJ replay.
186      - [ ] Retention sweep in place; GEMINI_API_KEY` rotated; allowed_video_root` set.
187      - [ ] CI green (L2 contract + L4 regression); coverage ? floor; $9 tracker all
?/?-resolved.
188
189      ## 12. References
190      Engine code [verified this session]`:
backend/main.py:12-17,42,52,307-328,331-340,342-396,638`;
backend/api/uploads.py:64,89,123,147`; backend/api/job_worker.py:37`;
backend/api/static/app.js:88,344,373`;
backend/storage/{base.py,local.py:33,ref.py,__init__.py:43}`;
backend/config/models.py:129,183,298`;
backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86,
gemini.py:470-485, motion.py:191-198}`; backend/pipeline/detectors/{native_wasb_runner.py,
stub_runner.py}`; backend/worker/__main__.py:88,98,149`; config.json` (default_segmenter:
gemini`). Engine docs: HOSTING_PLAN.md:42,102-108`; VIDEO_LOCALITY_MODEL.md`;
DESKTOP_APP_PLAN.md`; SETUP_NEW_MACHINE.md`; PLATFORM_ARCHITECTURE.md`;
DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; UI_PROPOSAL.md:41-66`;
PROJECT_DOC_TEMPLATE.md`. This repo: PER_RALLY_SELECTION.md`, REQUEST_ACCESS_FORM.md`,
PORTABILITY.md`, web/README.md`.
191
192      ---
193      ### Changelog
194      - 2026-06-20 ? Initial DRAFT`. Consolidated three single-box-appliance designs
(FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin
spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor

```

re-verified against source before writing (catching the native-WASB partial-wiring +  
`ai\_max\_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local),  
multi-provider GPU, and first-class Observability directives.

195

#### 14. assistant (2026-06-21T13:52:26.785Z)

I have strong grounding now. Let me check the worker entry point and main server bind, then load  
WebSearch.

#### 15. user (2026-06-21T13:52:28.145Z)

```
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local:``/``
9      is an identity passthrough; ``s3:``/`` / ``gcs:``/`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import csv
24     import json
25     import logging
26     import os
27     import sys
28     import tempfile
29     from typing import Any, List
30
31     from backend import storage
32     from backend.compute import ComputeJobSpec, get_compute_backend
33     from backend.config import (
34         StartupCheckError,
35         enforce_startup_checks,
36         load_config,
37         probe_cuda_available,
38     )
39     from backend.infrastructure.database import Database
40     from backend.pipeline.segmenters.base import segmenter_registry
41     from backend.pipeline.validators.base import registry as validator_registry
42     from backend.reporting import emit_indexer_report
43     from backend.results import Failure
44     from backend.utils.hashing import compute_video_id
45
46     logger = logging.getLogger("backend.worker")
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
50         str."""
51         low = v.lower()
52         if low in ("true", "false"):
```

```

49         return low == "true"
50     for cast in (int, float):
51         try:
52             return cast(v)
53         except ValueError:
54             pass
55     return v
56
57
58     def _apply_overrides(config: Any, sets: List[str]) -> None:
59         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
60         for kv in sets or []:

```

## 16. assistant (2026-06-21T13:52:31.621Z)

Now let me run the web searches in parallel to gather load-bearing facts about Docker realities.

## 17. user (2026-06-21T13:52:38.652Z)

Web search results for query: "micromamba in Docker image two python environments multi-stage build best practice"

Links: [{"title": "Using a multi-stage build to add micromamba functionality to an existing image · mamba-org/micromamba-docker · Discussion #175", "url": "https://github.com/mamba-org/micromamba-docker/discussions/175"}, {"title": "Reduction of image size with multi-staging Dockerfile · mamba-org/micromamba-docker · Discussion #412", "url": "https://github.com/mamba-org/micromamba-docker/discussions/412"}, {"title": "Advanced Usages ? micromamba-docker 2.6.0 documentation", "url": "https://micromamba-docker.readthedocs.io/en/latest/advanced\_usage.html"}, {"title": "Quick Start ? micromamba-docker 2.8.1 documentation", "url": "https://micromamba-docker.readthedocs.io/en/latest/quick\_start.html"}, {"title": "Docker image size compared to alternative package manager pip · Issue #2390 · mamba-org/mamba", "url": "https://github.com/mamba-org/mamba/issues/2390"}, {"title": "mambaorg/micromamba - Docker Image", "url": "https://hub.docker.com/r/mambaorg/micromamba"}, {"title": "Advanced Usages ? micromamba-docker 2.8.1 documentation", "url": "https://micromamba-docker.readthedocs.io/en/stable/advanced\_usage.html"}, {"title": "Shipping conda environments to production using pixi | QuantCo Engineering Blog", "url": "https://tech.quantco.com/blog/pixi-production/"}, {"title": "How to use in multi-stage builds? · Issue #10 · mamba-org/micromamba-docker", "url": "https://github.com/mamba-org/micromamba-docker/issues/10"}, {"title": "Activating another micromamba environment upon entry · mamba-org/micromamba-docker · Discussion #312", "url": "https://github.com/mamba-org/micromamba-docker/discussions/312"}]

Based on the search results, here's what I found about using micromamba in Docker images with two Python environments and multi-stage builds:

## Creating Multiple Python Environments

For most use cases you will only want a single conda environment within your derived image, but you can create multiple conda environments by using separate environment files with micromamba create commands.

## Multi-Stage Build Best Practices

By using a multi-stage build, you are able to copy the scripts directly from the mambaorg/micromamba image. This approach offers several advantages:

If an updated version of mambaorg/micromamba is released, you only need to update the mambaorg/micromamba tag within your downstream Dockerfile and it will pickup the newer micromamba binary along with any changes to the included scripts.

## Image Size Optimization



Instead of installing all libraries, you can copy `/opt/conda/lib/python3.11/site-packages` (all installed lib files) to the second stage of the image to help reduce image size. However, With multi-stage builds, you don't need to remove the cache directory, as it is not copied to the final image.

## Important Considerations

When working with micromamba in Docker builds, keep in mind that no conda environment is automatically activated during the execution of RUN commands within a Dockerfile, so you must set `ARG MAMBA_DOCKERFILE_ACTIVATE=1` to activate the conda environment.

The search results include practical examples of multi-stage builds using micromamba for managing environment size and improving maintainability.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

### 18. user (2026-06-21T13:52:38.913Z)

Web search results for query: "nvidia-container-toolkit Docker GPU passthrough --gpus all requirements 2025"

Links: [{"title": "GPU access | Docker Docs", "url": "https://docs.docker.com/engine/containers/gpu/"}, {"title": "GitHub - NVIDIA/nvidia-container-toolkit: Build and run containers leveraging NVIDIA GPUs · GitHub", "url": "https://github.com/NVIDIA/nvidia-container-toolkit"}, {"title": "Installing the NVIDIA Container Toolkit ? NVIDIA Container Toolkit", "url": "https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html"}, {"title": "Specialized Configurations with Docker ? NVIDIA Container Toolkit", "url": "https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/docker-specialized.html"}, {"title": "How to Install Docker on Debian for GPU Passthrough and Transcoding | DiyMediaServer", "url": "https://diymediaserver.com/post/install-docker-debian-gpu-passthrough-transcoding/"}, {"title": "Running a Sample Workload ? NVIDIA Container Toolkit", "url": "https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/sample-workload.html"}, {"title": "Nvidia GPU Passthrough in Unprivileged LXC Containers and Docker: A Step-by-Step Guide - Hostbor - Tech Reviews, Home Labs & AI Computing Guide", "url": "https://hostbor.com/gpu-passthrough-in-lxc-containers/"}]

Based on the search results, here's what you need to know about NVIDIA Container Toolkit and Docker GPU passthrough with the `--gpus all` flag:

## Installation Requirements

You need to install the NVIDIA GPU driver for your Linux distribution, and install the proprietary NVIDIA driver and the NVIDIA Container Toolkit on Debian.

## Configuration

Configure the container runtime by using the `nvidia-ctk` command to modify the `/etc/docker/daemon.json` file so that Docker can use the NVIDIA Container Runtime.

## Running Containers with GPU Access

Use `docker run -it --gpus all` or the Compose `device_requests` block, and the toolkit handles the driver mapping and permissions for you.

## Testing Your Setup

Run `docker run --rm --gpus=all nvidia/cuda nvidia-smi` to test?you want to see your card listed.

## Advanced Configuration

You can also specify specific driver capabilities. The `NVIDIA_DRIVER_CAPABILITIES` controls which driver libraries/binaries will be mounted inside the container, allowing you to specify capabilities like compute and utility.

The search results provide current information about this setup, with sources dated as recently as March 2026.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

### 19. user (2026-06-21T13:52:39.291Z)

Web search results for query: "Docker Desktop Windows GPU WSL2 nvidia-container-toolkit --gpus all support"

Links: [{"title":"GPU support in Docker Desktop for Windows | Docker Docs","url":"https://docs.docker.com/desktop/features/gpu/"},"{"title":"WSL 2 GPU Support for Docker Desktop on NVIDIA GPUs | Docker","url":"https://www.docker.com/blog/wsl-2-gpu-support-for-docker-desktop-on-nvidia-gpus/"},"{"title":"CUDA on WSL User Guide ? CUDA on WSL 13.3 documentation","url":"https://docs.nvidia.com/cuda/wsl-user-guide/index.html"},"{"title":"Guide to run CUDA + WSL + Docker with latest versions (21382 Windows build + 470.14 Nvidia) - CUDA on Windows Subsystem for Linux - NVIDIA Developer Forums","url":"https://forums.developer.nvidia.com/t/guide-to-run-cuda-wsl-docker-with-latest-versions-21382-windows-build-470-14-nvidia/178365"},"{"title":"How to Set Up Docker with WSL2 on Windows","url":"https://oneuptime.com/blog/post/2026-01-16-docker-wsl2-windows/view"},"{"title":"Nvidia Docker on WSL2. In 2022 if we intend to build? | by Frank Chung | DeepQ Research Engineering Blog | Medium","url":"https://medium.com/htc-research-engineering-blog/nvidia-docker-on-wsl2-f891dfe34ab"}]

Based on the search results, here's what I found about Docker Desktop Windows GPU support with WSL2 and nvidia-container-toolkit:

## GPU Support Overview

Docker Desktop for Windows supports NVIDIA GPU Paravirtualization (GPU-PV) on NVIDIA GPUs, allowing containers to access GPU resources for compute-intensive workloads like AI, machine learning, or video processing. GPU support in Docker Desktop is only available on Windows with the WSL2 backend.

## Requirements

To enable WSL 2 GPU Paravirtualization, you need a Windows machine with an NVIDIA GPU, up to date Windows 10 or Windows 11 installation, and up to date drivers from NVIDIA supporting WSL 2 GPU Paravirtualization.

## Using --gpus all Flag

The `--gpus=all` flag is available for the docker run command. You can test GPU access with a command like: ``docker run --rm -it --gpus=all nvcr.io/nvidia/k8s/cuda-sample:nbody nbody -gpu -benchmark``

## nvidia-container-toolkit Installation

To enable GPU support, the nvidia-container-toolkit needs to be installed in your WSL2 environment. The typical installation involves adding NVIDIA's repository and installing the ``nvidia-docker2`` package, which provides the necessary runtime support for the ``--gpus`` flag to work with Docker containers.

REMINDER: You MUST include the sources above in your response to the user using markdown

hyperlinks.

## 20. assistant (2026-06-21T13:52:41.701Z)

Now the key Windows-friction and image-size / compose realities.

## 21. user (2026-06-21T13:52:48.056Z)

Web search results for query: "Docker Desktop Windows licensing requires paid subscription large company free personal use 2025"

Links: [{"title": "Docker Desktop license agreement | Docker Docs", "url": "https://docs.docker.com/subscription/desktop-license/"}, {"title": "New Docker Licensing Requirements ? ICS Support Blog", "url": "https://sblog.ics.uci.edu/new-docker-licensing-requirements/"}, {"title": "Docker Plans FAQs | Docker", "url": "https://www.docker.com/pricing/faq/"}, {"title": "Docker License Changes in 2026", "url": "https://www.knowledgehut.com/blog/devops/docker-license-change"}, {"title": "License on Windows - General - Docker Community Forums", "url": "https://forums.docker.com/t/license-on-windows/129465"}, {"title": "Subscription | Docker Docs", "url": "https://docs.docker.com/subscription/"}, {"title": "Install Docker Desktop on Windows | Docker Docs", "url": "https://docs.docker.com/desktop/setup/install/windows-install/"}, {"title": "Pricing | Docker", "url": "https://www.docker.com/pricing/"}, {"title": "Docker Desktop and Windows ? what's the best option for you? | Microsoft Community Hub", "url": "https://techcommunity.microsoft.com/blog/itopstalkblog/docker-desktop-and-windows-%E2%80%93-what%E2%80%99s-the-best-option-for-you/3851656/replies/3852933"}]

Based on the web search results, here's what I found about Docker Desktop Windows licensing requirements in 2025:

### Licensing Model

Docker Desktop is licensed under the Docker Subscription Service Agreement. For teams using Docker Desktop for Windows or macOS, a subscription is now required for any use beyond personal or educational purposes.

### Free Usage Options

Docker Desktop is free for small businesses (fewer than 250 employees AND less than \$10 million in annual revenue), personal use, education, and non-commercial open source projects.

### Paid Subscription Requirements

Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR more than \$10 million USD in annual revenue) requires a paid subscription. The Docker Pro, Team, and Business subscriptions include commercial use of Docker Desktop.

### Additional Restrictions

Paid subscriptions are also required for government entities.

In summary, Docker Desktop for Windows requires a paid subscription for large companies, while personal use and small businesses meeting the criteria (under 250 employees and under \$10 million in revenue) can use it for free, as long as it's not for commercial purposes beyond the defined thresholds.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

## 22. user (2026-06-21T13:52:49.390Z)

Web search results for query: "Docker image size CUDA pytorch nvidia/cuda runtime base image GB pip install torch large"

Links: [{"title": "Optimizing PyTorch Docker images: how to cut size by 60% | Marton Veges", "url": "https://mveg.es/posts/optimizing-pytorch-docker-images-cut-size-by-60percent/"}, {"title": "CUDA dependency make Docker Image bloated - deployment - PyTorch Forums", "url": "https://discuss.pytorch.org/t/cuda-dependency-make-docker-image-bloated/164784"}, {"title": "[RFC] Reduce TorchServe Docker Image Size · Issue #2411 · pytorch/serve", "url": "https://github.com/pytorch/serve/issues/2411"}, {"title": "Optimizing Docker Setup for PyTorch Training with CUDA 12.8 and Python 3.11", "url": "https://www.runpod.io/articles/guides/docker-setup-pytorch-cuda-12-8-python-3-11"}, {"title": "How to reduce the Nvidia Docker image size ? Loki's Wager", "url": "https://lokiwager.github.io/posts/reduce-image-size/"}, {"title": "Image Layer Details - pytorch/pytorch:2.0.0-cuda11.7-cudnn8-devel", "url": "https://hub.docker.com/layers/pytorch/pytorch/2.0.0-cuda11.7-cudnn8-devel/images/sha256-96ccb2997a131f2455d70fb78dbb284baf4529aaf265e344bae932c8b32b2a4"}, {"title": "pytorch/pytorch - Docker Image", "url": "https://hub.docker.com/r/pytorch/pytorch"}, {"title": "Image Layer Details - pytorch/pytorch:1.7.0-cuda11.0-cudnn8-devel", "url": "https://hub.docker.com/layers/pytorch/pytorch/1.7.0-cuda11.0-cudnn8-devel/images/sha256-837e6964e5db6e5b35f4d5e98e9cac073ab757766039b9503f39c14beafb0e98"}]

I found several relevant search results about Docker image sizes with CUDA and PyTorch. Here's what the sources reveal:

## Image Size Overview

A basic Dockerfile with CUDA runtime, Python, and PyTorch can result in a staggering 7.6 GB image. The size varies significantly depending on the base image and configuration choices.

## Key Size Contributors

PyTorch comes with CUDA bundled, making the total size of the torch package quite large, around 2.5G. When pip caches this, it adds an extra 2.5 gigs of unused space.

## Base Image Comparisons

- **Runtime vs Base**: Using nvidia/cuda:11.8.0-base-ubuntu22.04 instead of the runtime version results in an image size of 5.6G, which is 26% lower.
- **Development Images**: The devel base image is significantly larger at 5 GB compared to the 800 MB of the base image.

## Optimization Results

Using pip3 with --no-cache-dir flag and the correct PyTorch index URL can reduce image size to just 2.9 GB, achieving a 62% reduction from the original size.

The search results show that final image sizes typically range from **2.9 GB** (optimized) to **7.6 GB** (unoptimized), with the nvidia/cuda base image offering better size efficiency than the runtime variant.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

### 23. user (2026-06-21T13:52:53.818Z)

Web search results for query: "docker-compose volumes bind mount Windows host path SQLite performance WSL2 filesystem slow"

Links: [{"title": "Improve disk performance", "url": "https://code.visualstudio.com/remote/advancedcontainers/improve-performance"}, {"title": "Docker with WSL2 on has bad performance with mounted local volumes · Issue #10476 · docker/for-win", "url": "https://github.com/docker/for-win/issues/10476"}, {"title": "Docker on Windows/WSL2 is slow: fixes that actually help ? cr0x.net", "url": "https://cr0x.net/en/docker-wsl2-performance-fixes/"}], [{"title": "Docker Desktop"}]

2.3.02 host volume mount performance · Issue #6742 · docker/for-win", "url": "https://github.com/docker/for-win/issues/6742"}, {"title": "Running docker/docker-compose from windows when referenced volume is in WSL2 does not appear to work · Issue #7742 · docker/for-win", "url": "https://github.com/docker/for-win/issues/7742"}, {"title": "Increase WSL2 and Docker Performance on Windows By 20x | by Suyash Singh | Medium", "url": "https://medium.com/@suyashsingh.stem/increase-docker-performance-on-windows-by-20x-6d2318256b9a"}, {"title": "Docker Desktop: WSL 2 Best practices | Docker", "url": "https://www.docker.com/blog/docker-desktop-wsl-2-best-practices/"}]

Here are the search results for your query about Docker Compose volumes, bind mounts, Windows host paths, SQLite performance, and WSL2 filesystem:

## Key Findings:

### **\*\*Performance Issues with Windows Host Paths:\*\***

If you mount files that live in the Windows file system, you won't get performance benefits, as /mnt/c is actually a mountpoint exposing Windows files through a Plan9 file share. Bind mounts from Windows paths are often the slow ones.

### **\*\*WSL2 Filesystem Performance:\*\***

If you store your source code in the WSL 2 filesystem, you will see improved performance along with better compatibility for things like setting permissions. Bind mounts from WSL ext4 can be fine.

### **\*\*Database Performance Specifically:\*\***

High latency on simple queries with lots of disk writes and occasional stalls can occur when the DB data directory is on a bind mount crossing the Windows boundary; the fix is to store DB data on a named volume.

### **\*\*Alternative Solutions:\*\***

Since macOS and Windows run containers in a VM, bind mounts are not as fast as using the container's filesystem directly; Docker has the concept of a local "named volume" that can act like the container's filesystem but survives container rebuilds, making it ideal for storing data folders where write performance is critical.

The consensus is that for SQLite databases and other I/O-intensive operations, using named volumes instead of bind mounts to Windows paths, or storing your code in the WSL2 filesystem instead of Windows filesystem, yields significantly better performance.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

## 24. assistant (2026-06-21T13:52:57.362Z)

A few more targeted searches on alternatives and the dual-torch GPU-sharing reality, plus WASB conda CUDA 11.3 compatibility with newer drivers.

## 25. user (2026-06-21T13:53:05.626Z)

Web search results for query: "single docker image multiple python versions vs separate containers microservice tradeoffs ML inference worker"

Links: [{"title": "Docker Magic: Installing and Managing Multiple Python Versions | by Padmajeet Mhaske | Medium", "url": "https://mhaske-padmajeet.medium.com/docker-magic-installing-and-managing-multiple-python-versions-00c45868c045"}, {"title": "Running multiple projects with different python versions/ray versions and docker images - Ray Core - Ray", "url": "https://discuss.ray.io/t/running-multiple-projects-with-different-python-versions-ray-versions-and-docker-images/6253"}, {"title": "GitHub - dhermes/python-multi: Single purpose repository for a Docker image with multiple Python versions installed · GitHub", "url": "https://github.com/dhermes/python-multi"}, {"title": "Containerization Best Practices: Python Dockerfile Multi-Stage Builds for Slim Images", "url": "https://johal.in/containerization-best-practices-python-dockerfile-multi-stage-builds-for-slim-images/"}], [{"title": "Enabling Multiple Python Versions in a Single Docker"}]

```
Container and Managing Them with Pipenv | by Padmajeet Mhaske | Medium", "url": "https://mhaske-padmajeet.medium.com/enabling-multiple-python-versions-in-a-single-docker-container-and-managing-them-with-pipenv-c94227f90c69"}, {"title": "Slimming Down Your Docker Images: A Guide to Single-Stage vs. Multi-Stage Python Builds - DEV Community", "url": "https://dev.to/aakashkhanna/slimming-down-your-docker-images-a-guide-to-single-stage-vs-multi-stage-python-builds-3m77"}, {"title": "Docker image with two different python versions - Docker Desktop - Docker Community Forums", "url": "https://forums.docker.com/t/docker-image-with-two-different-python-versions/139998"}, {"title": "Optimizing Image Size: Multi-Stage Builds", "url": "https://apxml.com/courses/docker-for-ml-projects/chapter-5-containerizing-ml-inference/optimizing-image-size-multistage"}]]
```

Based on the search results, here's what I found about managing Python versions in Docker for ML inference workers:

## Single Docker Image with Multiple Python Versions

A Dockerfile itself cannot directly contain multiple Python versions, as each Docker image is typically based on a single operating system environment with a single version of Python installed. However, there are workarounds:

- You can create a Docker image that includes multiple Python versions by manually installing them and configuring your environment accordingly.
- Some base images, like jessestuart/multi-python, are specifically designed to include multiple Python versions, and you can use such an image as your base and then configure your Dockerfile to use the desired Python version.
- You can use pyenv to manage multiple Python versions within a Docker container, though this approach is more flexible but requires additional setup.

## Microservice Architecture Considerations

For ML inference workers, the search results show practical deployment patterns. Companies deploying numerous models to production use docker containers and Amazon ECR, where each docker container talks to an internal pipeline orchestrator and picks the next task to work on.

## Image Optimization for ML Inference

Rather than bundling multiple Python versions, the industry trend is toward optimized containerization: Multi-stage builds cut Python image sizes by 75%, making Kubernetes deploys faster?essential for microservices in 2025. Removing build tools, temporary files, and unnecessary libraries can drastically shrink the final image size, leading to faster deployments and lower storage costs.

The practical recommendation for ML inference workers appears to be using separate optimized containers per Python version rather than bundling multiple versions in a single image.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

### 26. user (2026-06-21T13:53:05.852Z)

Web search results for query: "CUDA 11.3 torch 1.11 container run on newer NVIDIA driver CUDA 12 backward compatibility minor version"

Links: [{"title": "CUDA Compatibility ? CUDA Compatibility", "url": "https://docs.nvidia.com/deploy/cuda-compatibility/latest/"}, {"title": "Forward Compatibility ? CUDA Compatibility", "url": "https://docs.nvidia.com/deploy/cuda-compatibility/forward-compatibility.html"}, {"title": "CUDA Compatibility Release r595 NVIDIA Corporation Mar 31, 2026", "url": "https://docs.nvidia.com/deploy/pdf/CUDA\_Compatibility.pdf"}, {"title": "Minor Version Compatibility ? CUDA Compatibility", "url": "https://docs.nvidia.com/deploy/cuda-compatibility/minor-version-compatibility.html"}, {"title": "Frequently Asked Questions ? CUDA Compatibility", "url": "https://docs.nvidia.com/deploy/cuda-compatibility/faq.html"}]]

```
Compatibility","url":"https://docs.nvidia.com/deploy/cuda-compatibility/frequently-asked-questions.html"},"{"title":"CUDA CONNUNDRUM on COMPATIBILITY AND FURTHER DISCUSSIONS on DRIVERS & CUDA TOOLKITS | by Biswal Anwesh | Medium","url":"https://medium.com/@biswal.anwesh29/cuda-connundrum-on-compatibility-and-further-discussions-on-drivers-cuda-toolkits-5391330c8916"},"{"title":"Are newer CUDA versions in a major series (11.x, 12.x) backwards compatible with another? - CUDA Setup and Installation - NVIDIA Developer Forums","url":"https://forums.developer.nvidia.com/t/are-newer-cuda-versions-in-a-major-series-11-x-12-x-backwards-compatible-with-another/269535"},"{"title":"Does cuda container support backwards compatibility? · Issue #940 · NVIDIA/nvidia-container-toolkit","url":"https://github.com/NVIDIA/nvidia-container-toolkit/issues/940"},"{"title":"Nvidia driver and CUDA version compatibility chart · GitHub","url":"https://gist.github.com/rlan/258b7c030364735be10c9df277cff5ed"}]]
```

Based on the search results, here's what I found about running CUDA 11.3 with PyTorch 1.11 containers on newer NVIDIA drivers with CUDA 12:

## Backward Compatibility Overview

CUDA Compatibility includes Minor Version Compatibility, available starting with CUDA 11, which allows applications built within the same major CUDA release family to run on a sufficiently new driver, with some feature limitations.

## Your Specific Scenario (CUDA 11.3 ? CUDA 12 Driver)

Unfortunately, running a CUDA 11.3 container on a CUDA 12 driver involves crossing major version boundaries, which is more complex:

- 11.x and 12.x applications will be supported due to backward compatibility and future 12.x applications will be supported due to minor-version compatibility.

- Drivers have always been backwards compatible with CUDA. This means that a CUDA 11.0 application will be compatible with R450 (11.0), R455 (11.1) and beyond.

## Container-Specific Considerations

CUDA minor version compatibility and CUDA forward compatible upgrade both work when using either NGC Deep Learning Framework containers or using containers that are based on the official CUDA base images. The images include the CUDA compatible upgrade libraries and the NVIDIA Container Toolkit (nvidia-docker2) has logic to correctly load the required libraries.

However, if you see "compatibility mode is UNAVAILABLE": Cause: Host driver is too old (e.g., 418.126) for the container's CUDA version. Solution: Upgrade the host driver or use a container built for older CUDA.

The key takeaway is that CUDA 11.3 applications should generally work on CUDA 12 drivers due to backward compatibility, but ensure your container images and driver setup properly support this compatibility mode.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

## 27. user (2026-06-21T13:53:07.111Z)

Web search results for query: "Docker Compose GPU device\_requests reservations devices driver nvidia count all syntax"

Links: [{"title":"Run Docker Compose services with GPU access | Docker Docs","url":"https://docs.docker.com/compose/how-tos/gpu-support/"},"{"title":"Assigning Specific GPUs to Docker Containers with Docker Compose | by Ramkrushna Maheshwar | Medium","url":"https://medium.com/@maheshwar.ramkrushna/assigning-specific-gpus-to-docker-containers-with-docker-compose-1fd34eeaae91"},"{"title":"Docker compose GPU device selection as no effect - General - Docker Community Forums","url":"https://forums.docker.com/t/docker-

```
compose-gpu-device-selection-as-no-effect/140217"}],{"title":"Docker Compose GPU Access and Reservation | Medium","url":"https://medium.com/@TheHaseebHassan/docker-compose-gpu-access-e177cd4ff600"}, {"title":"Specialized Configurations with Docker ? NVIDIA Container Toolkit","url":"https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/docker-specialized.html"}, {"title":"GPU support in Docker Desktop for Windows | Docker Docs","url":"https://docs.docker.com/desktop/features/gpu/"}, {"title":"What is the latest proper way to use the Nvidia Container Toolkit with docker compose? - Compose - Docker Community Forums","url":"https://forums.docker.com/t/what-is-the-latest-proper-way-to-use-the-nvidia-container-toolkit-with-docker-compose/144729"}, {"title":"How to Run Docker Compose Containers With GPU Access","url":"https://www.howtogeek.com/devops/how-to-run-docker-compose-containers-with-gpu-access/"}]]
```

Based on the search results, here's the comprehensive information about Docker Compose GPU device\_requests syntax:

## Docker Compose GPU Device Reservations Syntax

GPUs are referenced in a compose.yaml file using the device attribute from the Compose Deploy specification, within your services that need them.

### Key Device Properties

The device structure supports the following configurable properties:

1. **capabilities** - This value is specified as a list of strings. For example, capabilities: [gpu]. You must set this field in the Compose file. Otherwise, it returns an error on service deployment.
2. **count** - Specified as an integer or the value all, represents the number of GPU devices that should be reserved (providing the host holds that number of GPUs).
3. **driver** - Specified as a string, for example driver: 'nvidia'
4. **device\_ids** - A list of strings representing GPU device IDs from the host.
5. **options** - Key-value pairs representing driver specific options.

### Important Constraints

You can use count or device\_ids in each of your service definitions. An error is returned if you try to combine both, specify an invalid device ID, or use a value of count that's higher than the number of GPUs in your system.

### Default Behavior

If no device\_ids are set, all GPUs available on the host are used by default.

### Example Configuration

```
services: test: image: nvidia/cuda:12.9.0-base-ubuntu22.04 command: nvidia-smi deploy: resources: reservations: devices: - driver: nvidia count: 1 capabilities: [gpu]
```

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

### 28. assistant (2026-06-21T13:53:10.618Z)

I have enough to synthesize. Let me do one final check on the engine's server bind and a verification of the `backend/api/static` dead-UI claim, plus confirm `backend.main` server flags.

### 29. user (2026-06-21T13:53:12.564Z)



```
562:     parser.add_argument("--server", action="store_true", help="Run the FastAPI local API server")
563:     parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI server on")
735:         uvicorn.run(app, host="127.0.0.1", port=args.port)
```

### 30. user (2026-06-21T13:53:12.849Z)

```
backend\api\static\index.html
backend\api\static\style.css
backend\api\static\app.js
```

### 31. assistant (2026-06-21T13:53:18.819Z)

I have verified the load-bearing engine facts and gathered the Docker realities. Let me confirm the server hard-binds to 127.0.0.1 (line 735) ? that's a real constraint for any container approach. Now I'll synthesize the research into the structured output.

Key findings grounded:

- Engine server hard-binds `host="127.0.0.1"` at `backend/main.py:735` ? inside a container this would NOT be reachable from the host; the appliance needs `0.0.0.0` bind or a flag.
- The existing `deploy/cloudrun/Dockerfile` already proves the two-Python-env-in-one-image pattern (CUDA 12.4.1 base + miniconda `wasb` env @ torch 1.11+cu113 as a subprocess via `\$WASB\_PYTHON`).
- WASB weights deliberately NOT baked (WASB-C3 licensing) ? must mount/fetch at runtime.

### 32. user (2026-06-21T13:55:07.833Z)

Structured output provided successfully

### 33. assistant (2026-06-21T13:55:18.908Z)

Research complete. The structured output has been returned to the orchestrator.

Summary of what I grounded and found:

**\*\*Verified engine facts\*\*** (read against source, not assumed):

- The two-Python-envs-in-one-image pattern is **\*\*already shipped\*\*** in `deploy/cloudrun/Dockerfile:18,32-57` (CUDA 12.4.1 base + Miniconda `wasb` env @ torch 1.11+cu113, reached via `\$WASB\_PYTHON` subprocess). The torches deliberately never co-install (Dockerfile:8-12).
- WASB weights are **\*\*not baked\*\*** (WASB-C3 licensing) ? mandatory runtime mount (Dockerfile:14-16,56).
- Server **\*\*hard-binds `127.0.0.1`\*\*** at `backend/main.py:735` ? inside a container this is unreachable from the host; the appliance forces a `0.0.0.0` bind with the firewall/edge gate as the safety boundary.
- `torch` is intentionally out of `pyproject.toml` (PEP621 can't express `--index-url`) ? CPU vs GPU images diverge on one torch line.
- Worker is in-process (`ThreadPoolExecutor(max\_workers=1)`, per LOCAL\_APPLIANCE\_ARCHITECTURE.md D3) ? one-service container is the honest unit; splitting WASB into its own container is net-new RPC plumbing.

**\*\*Recommendation:\*\*** two-variant strategy ? a small CPU-only default image (covers the Gemini/stub/motion CUIs a no-GPU coach actually runs) plus an opt-in `gpu` compose profile that extends the existing cloudrun Dockerfile (micromamba over Miniconda). SQLite in a named volume; videos/weights on WSL2-side mounts, never `/mnt/c`. Flagged the honest fork against owner decision D10 (Docker Desktop + WSL2-GPU is itself friction for a non-tech Windows user).

Relevant file paths: `C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\cloudrun\Dockerfile`, `C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\digitalocean\gpu\_setup.sh`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py` (line 735 bind), `C:\Users\avidu\Projects\badminton-highlight-indexer\pyproject.toml`,  
`C:\Users\avidu\Projects\kheIsutra\docs\LOCAL\_APPLIANCE\_ARCHITECTURE.md`.